Paul Struble
3038299344
Special Participation E

ChatGPT State Space Models Tutoring Using Study & Learn Mode on GPT 5.1

Executive Summary

In this interaction, I use the ChatGPT “Study and Learn” mode (on GPT 5.1) to interact with the LLM about State Space Models (SSMs). The model walks me through the lecture notes (which I attach in the first message as PDFs) while testing my intuition/understanding along the way and adjusting its course/conversation according to my responses. In this document, I provide some observations/analysis about the model in the “Study and Learn” mode.

Link to Conversation: https://chatgpt.com/share/693a7636-a964-8007-9fb7-ad29c14ccedb

Model Details:

- Model: ChatGPT 5.1
- Used ChatGPT web frontend (chatgpt.com)
- Disabled all personalization features
- Used “default” personality

Lecture 16.pdf
PDF

Lecture 17.pdf
PDF

You are an expert machine learning tutor helping me to study the material in my deep learning / neural networks course. Read through the attached Lecture 16 and Lecture 17 material on State Space Models (SSMs). Help me study the material on SSMs by walking me through the content in the lectures, asking me questions to test my understanding, and quizzing me. Adhere to only the material covered in the lecture notes (and very closely related knowledge if appropriate). Assume that I am already familiar with the material and would like to deepen my understanding and build a stronger intuition.

Great — since you already know the material and want to *deepen intuition*, let's make this a guided study session rather than a lecture.
Before we dive in, I need **one quick calibration question** (per the study-mode rules):

⭐ What's your current comfort level with SSMs?

Which of the following best describes you?

A. *"I understand the basic idea: linear recurrence + convolution view, but I want deeper intuition."*
B. *"I mostly get S4/Mamba but want to clean up some fuzzy spots."*
C. *"I understand the control-theory formulation, but I want intuition for why this helps with long-range dependencies."*
D. Something else — describe in a sentence.

Pick one, and then I'll start walking you through Lecture 16→17 with active questioning.

To preview where we'll go (using your lecture notes)

- **Lecture 16** (pages 1–4) motivates SSMs as *linear RNNs with no horizontal nonlinearity* and shows why this turns sequence processing into *parallelizable convolutions*. Lecture 16
- Later pages (6–11) illustrate how each output channel becomes a convolution with filters derived from A, B, C.
- **Lecture 17** builds intuition about *long-range memory*: eigenvalues of A, stability, HIPPO initialization, and continuous-time views leading to Mamba gating. Lecture 17

As we go, I'll:

- connect the discrete and continuous views,
- ask conceptual questions (one at a time),
- help you build intuition for memory, decay, and gating.

So—**which option (A/B/C/D) matches where you are?**

A matches where I am.

The model starts by gauging where my knowledge currently is, presumably to tailor the discussion accordingly. It seems to have effectively read the information from the provided PDFs of the lecture notes.

Great — since you're at **A**, we'll build intuition from the core ideas in Lecture 16 and gradually layer in the deeper insights from Lecture 17. I'll guide you with short explanations → a single question → your turn → then we continue.

Step 1 — Why removing the horizontal nonlinearity changes everything (Lecture 16 p.1–2)

Your notes highlight the key change:

Traditional RNN:

$$h_{t+1} = \sigma(Ah_t + Bx_t + b)$$

State-space version (SSM):

$$h_{t+1} = Ah_t + Bx_t$$

$$y_t = Ch_t + Dx_t$$

(See Lecture 16, pp.1–2: the boxed equations and note "just eliminate it!") Lecture 16

This makes the system a **linear time-invariant (LTI)** recurrence.
Once linear, we can *unroll* it exactly:

$$h_t = A^t h_0 + \sum_{k=0}^{t-1} A^{t-1-k} B x_k,$$

which means that **each output is a convolution of the input sequence with filters derived from powers of A.**

This is the key idea behind parallelization:
once the filters CA^kB are known, computing y_t becomes a **convolution**, which can be computed using FFT in $O(T \log T)$ (Lecture 16 p.3). Lecture 16

Question 1 — Warm-up intuition check

Why does *removing the nonlinearity* allow parallelization?
Choose the best explanation **in your own words**:

1. Because nonlinearities make the sequence depend on itself in a way that can't be unrolled.
2. Because in a linear recurrence, we can express h_t directly in terms of all past inputs using matrix powers, so the dependence is fixed and we can compute all outputs with convolution.
3. Because linear systems run faster on GPUs.

You can modify/expand your answer — I just want to hear *your* reasoning.

I think 2 is the best reasoning. Once we remove the nonlinearity, the update rule for the hidden state is just a linear operation with the same A and B matrices at every step. That means the hidden state's dependence on the past is predetermined by simple matrix powers. We can do a convolution over the entire sequence (efficiently with FFT) because it is LTI.

The model now begins a review to check my intuition. I engage with the model's dialogue and start by demonstrating a clear understanding of the material the model is covering and the questions it is asking.

Perfect — that's exactly the intuition Lecture 16 wants you to internalize.
Let's deepen it just a bit and then move forward.

Because the recurrence is **linear** *and* the parameters (A, B, C) are **time-invariant**, the whole system becomes:

- **shift-equivariant** (same behavior regardless of where in the sequence you start), and
- **composable through matrix powers** (the influence of an input at time k on output at time t is exactly $CA^{t-k-1}B$).

This is precisely why the convolution view in Lecture 16 pages 6–11 emerges so cleanly. Lecture 16

Step 2 — Why outputs become *many* convolutions (Lecture 16 p.6–10)

Your notes (e.g., page 7) show that each output channel $y_c[t]$ is:

$$y_c[t] = \sum_{\ell=0}^{t} F_{c,\ell}\, x[t-\ell],$$

where $F_{c,\ell} = C_c A^\ell B$.
This means:

- Each output dimension has its **own learned filter**.
- If input dim = d_x and output dim = d_y, you effectively have $d_x \times d_y$ filters (page 10).

This is why the lecture starts talking about structural constraints (diagonal/HiPPO/etc.) to keep things efficient.

Question 2 — Intuition check

Suppose A is diagonal (Lecture 16, p.5).
Why does making A diagonal drastically simplify computing the filters CA^kB?

Here are two hints — choose one (or combine them):

- **Hint 1:** For diagonal A, what does A^k look like?
- **Hint 2:** What operation does left-multiplying by C and right-multiplying by B become when A is diagonal?

Give me your intuitive explanation — no formulas needed unless helpful.

Making A diagonal makes each power of A just element-wise powers which is easy to compute.

The model's next response incorporates the response I gave in my previous message and returns its dialogue to the next part of the lecture notes to continue the intuition check. I intentionally make my response simpler/briefer here to see how the model responds.

Exactly — that's the essential insight. Let's unpack it one layer deeper, then I'll give you the next question.

When A is diagonal:

- $A = \text{diag}(\lambda_1, \lambda_2, \dots)$
- Then $A^k = \text{diag}(\lambda_1^k, \lambda_2^k, \dots)$

So each channel of the hidden state evolves **independently**, and computing A^k doesn't require any matrix multiplications — just raising scalars to powers.

This means the filters

$$F_k = CA^kB$$

become:

- "scale the columns of B" (elementwise),
- then "mix with C,"
  and it's all *simple, cheap*, and ***parallelizable***.

This is why Lecture 16 suggests diagonal A as the "easiest structure to experiment with" (p.5).
Lecture 16

Step 3 — Transition to Lecture 17: Long-range dependencies

Lecture 17 begins by recalling:

$$h_{t+1} = Ah_t + Bx_t, \quad y_t = Ch_t$$

and emphasizes that **all past information must fit inside h_t** (page 1).
So the question becomes:
How do the eigenvalues of A determine how long information persists?

Your notes (Lecture 17 p.2) summarize:

- $|\lambda| > 1$: explodes (bad)
- $|\lambda| < 1$: dies out quickly
- $|\lambda| = 1$: "critical damping" (ideal long memory) Lecture 17

This is the heart of long-range memory in SSMs.

Question 3 — Deep intuition check

Imagine one eigenvalue of A is very close to 1 (say $\lambda = 0.999$).
Why does that help the model capture long-range dependencies?

Give an intuitive explanation — think in terms of:

- how fast past information decays, and
- what that means for how long the system "remembers" things.

Your turn.

I am not completely sure, but my intuition is that an eigenvalue close to 1 means that the information in the direction of the corresponding eigenvector remains relatively constant over time since it doesn't explode or vanish (magnitude is roughly preserved). My simple intuition then is that this "preservation" of information acts as an effective form of memory.

The model continues similarly despite the shorter answer. The model's response to my answer is a little longer than before, but it doesn't seem that it changed much from the shorter response. Next, I intentionally give a response that is less confident and only reflects a higher-level intuition to see how the model responds.

Great — that's exactly the right starting intuition.
Let's connect it directly to SSM behavior in the simplest possible way, staying grounded in Lecture 17.

✅ Intuition: Complex eigenvalues = oscillatory memory traces

Suppose an eigenvalue of A is:

$$\lambda = re^{i\theta}$$

Then the recurrence in that eigen-direction evolves as:

$$\lambda^t = r^t e^{it\theta}.$$

This splits into two effects:

1. The magnitude r^t controls how fast the memory fades.

This is exactly the same role real eigenvalues play. If $r \approx 1$, it decays slowly ⇒ long memory.

2. The phase $e^{it\theta}$ produces repeated rotations (oscillations).

This is the key intuition:

> **The system doesn't just "store" a value — it transforms it over time in a cyclic / wave-like way.**

That means the memory isn't just a static trace; it can encode:

- periodic structures
- rhythmic dependencies
- patterns that depend on phase or relative position
- multiple time scales depending on different frequencies

And unlike real eigenvalues (which can only flip sign or decay), complex eigenvalues can encode **rich temporal signatures**.

This is exactly why Lecture 17 introduces complex eigenvalues as a solution to the too-rigid ±1 real case (p.3).

Lecture 17

🔍 The analogy that usually clicks

Think of each eigen-mode as a "memory channel."
Real eigenvalues give you:

- one channel that decays steadily (λ≈0.9)
- maybe one that flips sign (λ≈–1)

But complex eigenvalues give you:

- channels that **rotate**
- channels that **spin slower or faster**
- channels whose amplitude decays at a chosen rate

This is like letting your model store memories not just as "values," but as **waveforms**.

That dramatically expands expressivity for sequence modeling.

⭐ Question 5 — Quick check

Based on this explanation, how would you describe — in your own words — the *advantage* of oscillatory modes in an SSM?

Just a short sentence or two.

Oscillatory modes let the SSM store information as a decaying waveform instead of just a number which adds phase and frequency structure its memory. That allows for the model to represent time and seasonality more effectively.

The model acknowledges my high-level intuition and continues with a longer explanation/reasoning, so it seems to have responded to the looser answer I gave. It gives an analogy as well, seemingly to help me since my understanding from my previous answer was less confident.

Exactly — that's a clean and accurate intuition.

Let's lock that in:

> **Complex eigenvalues let an SSM store information as slowly decaying waves, not just decaying scalars.**
> **This gives the model access to phase, frequency, and periodic structure — far richer than real-valued decay alone.**

That is *exactly* why Lecture 17 motivates complex parameterizations of A.

Step 4 — Linking discrete SSMs to continuous-time intuition (Lecture 17 p.4–6)

Your notes introduce the continuous-time linear ODE:

$$\dot{h}(t) = Ah(t) + Bx(t) \qquad (\text{Lecture 17 p.4})$$

and then show how we discretize it to get the SSM used in ML:

$$h_{t+1} = \bar{A}h_t + \bar{B}x_t,$$

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = \left(\int_0^{\Delta} e^{As}\, ds\right) B$$

(p.5). Lecture 17

This continuous-time perspective gives two powerful intuitions:

1. Stability = eigenvalues with negative real part

If $\mathrm{Re}(\lambda) < 0$, solutions decay over time.
This parallels the discrete case where $|\bar{\lambda}| < 1$.

2. Continuous parameters make learning easier

Instead of directly learning $\bar{A}$ (which must satisfy stability constraints), we learn A in continuous-time.
Then $\bar{A} = e^{A\Delta}$ **automatically** satisfies the discrete stability constraints as long as $\mathrm{Re}(\lambda) < 0$.

This is why many SSM architectures (S4, Mamba) *parametrize A in continuous time*.

⭐ Question 6 — Conceptual check

Why might it be *easier* for gradient descent to learn stable long-range behavior by optimizing the continuous-time matrix A rather than directly optimizing the discrete-time matrix $\bar{A}$?

You can answer intuitively — no need for formulas.

I am not sure.

Here, I directly tell the model that I do not know the answer to see how it will respond.

Totally fine — this is one of the trickier conceptual leaps in Lecture 17.
Let's build the intuition step by step.

✅ Key idea

In **continuous time**, stability is controlled by a *simple, smooth condition*:

$$\mathrm{Re}(\lambda) < 0.$$

That's it.
Push eigenvalues a little more negative → more stability.
Push them less negative → longer memory.

Gradient descent can move these eigenvalues smoothly.

🔄 But in discrete time, the corresponding stability condition is:

$$|\bar{\lambda}| < 1.$$

This creates a problem:

1. The unit circle is a sharp boundary.

If $|\bar{\lambda}| > 1$: exploding.
If $|\bar{\lambda}| < 1$: decaying.
But there's no smooth "slope" around the boundary.

2. Small parameter changes can cause huge behavioral changes.

A tiny push across the unit circle flips the system from stable → unstable.
This makes optimization fragile.

🔄 But in discrete time, the corresponding stability condition is:

$$|\bar{\lambda}| < 1.$$

This creates a problem:

1. The unit circle is a sharp boundary.

If $|\bar{\lambda}| > 1$: exploding.
If $|\bar{\lambda}| < 1$: decaying.
But there's no smooth "slope" around the boundary.

2. Small parameter changes can cause huge behavioral changes.

A tiny push across the unit circle flips the system from stable → unstable.
This makes optimization fragile.

⭐ The solution: parameterize in continuous time

Because:

$$\bar{A} = e^{A\Delta},$$

the mapping

$$\mathrm{Re}(\lambda(A)) < 0 \quad \Longrightarrow \quad |\lambda(\bar{A})| < 1$$

holds automatically.

And since exponential is a smooth map:

- smoothly moving $\mathrm{Re}(\lambda(A))$
  ⇒ smoothly moves the modulus $|\lambda(\bar{A})|$

This *removes the sharp boundary* and makes training dramatically more stable.

This is exactly the motivation of Lecture 17 p.5–6.
Lecture 17

⭐ Try it now — Question 6 (simplified):

In your own words:

Why does learning A in continuous time make it easier to learn stable long-range behavior than learning $\bar{A}$ directly in discrete time?

Just summarize the idea in 1–2 sentences.

The model responds by giving a longer explanation since I did not know, and it then gives a simplified question to ensure I understand the concept before moving on to other material.